

GUÍA DE LABORATORIO

Fundamentos del desarrollo web

Semana 1 · Front-end, back-end, HTML, CSS, JavaScript y marcos de desarrollo

PRODUCTO DE APRENDIZAJE

Construirás una página web semántica, accesible y adaptable, con estructura HTML, presentación CSS y una interacción sencilla en JavaScript. Los archivos quedarán separados siguiendo buenas prácticas profesionales.

Curso	Marcos de Desarrollo Web
Semana	1
Modalidad	Laboratorio guiado
Duración sugerida	4 horas académicas
Nivel	Introductorio con enfoque profesional
Requisitos	Manejo básico del sistema operativo

APRENDER HACIENDO · COMPRENDER ANTES DE USAR UN FRAMEWORK

1. Orientación de la práctica

Esta guía introduce el flujo completo que sigue el navegador cuando recibe una página web. No busca memorizar etiquetas, sino comprender responsabilidades, organizar archivos y comprobar cada decisión mediante las herramientas del navegador.

1.1 Resultado de aprendizaje

AL FINALIZAR

Explica la diferencia entre front-end y back-end; reconoce la función de HTML, CSS y JavaScript; organiza un proyecto web básico; implementa una interfaz semántica y accesible; y verifica su funcionamiento con las herramientas de desarrollo del navegador.

1.2 Criterios de logro

- Distingue cliente, servidor, solicitud y respuesta sin confundir sus responsabilidades.
- Usa HTML semántico y una jerarquía correcta de encabezados.
- Separa estructura, presentación y comportamiento en archivos distintos.
- Aplica estilos adaptables sin depender de medidas rígidas.
- Implementa una interacción progresiva que no impide acceder al contenido.
- Valida el resultado, interpreta errores de consola y documenta evidencias.

1.3 Agenda sugerida

Bloque	Actividad	Tiempo
1	Fundamentos y arquitectura web	35 min
2	Preparación del entorno	25 min
3	Construcción guiada	100 min
4	Verificación y depuración	35 min
5	Reto y cierre	45 min

2. Fundamentos que debes comprender

2.1 ¿Qué ocurre cuando visitas una web?

El navegador actúa como cliente. Solicita un recurso mediante HTTP o HTTPS; un servidor procesa la solicitud y devuelve una respuesta. La respuesta puede contener HTML, CSS, JavaScript, imágenes o datos. El navegador interpreta esos recursos y construye la interfaz.

Flujo conceptual

Usuario → Navegador (cliente) → Solicitud HTTPS → Servidor
 Usuario ← Interfaz renderizada ← Respuesta HTML/CSS/JS ← Servidor

PRECISIÓN

Internet es la infraestructura de redes. La Web es uno de los servicios que funciona sobre Internet. HTTP es el protocolo de comunicación habitual de la Web.

2.2 Front-end y back-end

Aspecto	Front-end	Back-end
Dónde se ejecuta	Principalmente en el navegador	En un servidor
Responsabilidad	Interfaz, experiencia e interacción	Reglas de negocio, datos y seguridad
Tecnologías iniciales	HTML, CSS y JavaScript	Java, Spring Boot, SQL
Ejemplo	Mostrar un formulario	Validar y guardar una solicitud

Una aplicación real combina ambos lados. El front-end nunca debe ser la única barrera de seguridad: toda validación relevante debe repetirse en el servidor.

2.3 Las tres capas del navegador

Tecnología	Pregunta que responde	Responsabilidad
HTML	¿Qué contenido existe?	Estructura y significado
CSS	¿Cómo se presenta?	Diseño, distribución y adaptación
JavaScript	¿Cómo reacciona?	Comportamiento e interacción

BUENA PRÁCTICA

Primero crea contenido comprensible con HTML. Después mejora la presentación con CSS. Finalmente añade JavaScript solo cuando la interacción aporte valor. Este enfoque se denomina mejora progresiva.

2.4 ¿Qué es un framework?

Un framework proporciona convenciones, componentes y una estructura de trabajo para resolver problemas recurrentes. Reduce trabajo repetitivo, pero también introduce reglas y dependencias. Una biblioteca suele ofrecer funciones que el programa invoca; en un framework, gran parte del flujo está definido por el propio marco.

En este curso se estudiarán dos familias: Bootstrap para interfaces y Spring Boot para aplicaciones Java del lado servidor. Comprender la base evita utilizar componentes como “cajas negras”.

3. Preparación del entorno

3.1 Herramientas

- Visual Studio Code actualizado.
- Un navegador moderno con herramientas de desarrollo (Chrome, Edge o Firefox).
- Extensión opcional Live Server para recarga automática.
- Una carpeta local de trabajo. No es necesario instalar un servidor back-end en esta semana.

SEGURIDAD

No descargues extensiones de autores desconocidos. Revisa editor, cantidad de instalaciones, fecha de actualización y permisos solicitados.

PASO 1 Crear la carpeta del proyecto

Crea una carpeta llamada semana-01-web. Evita espacios, tildes y caracteres especiales en rutas de proyectos.

PASO 2 Abrir la carpeta completa

En Visual Studio Code selecciona Archivo > Abrir carpeta. Trabajar sobre la carpeta permite que las rutas relativas y las herramientas funcionen de forma predecible.

PASO 3 Crear la estructura

Crea las carpetas y archivos exactamente como se muestra. Los nombres se escriben en minúscula y con guion cuando contienen varias palabras.

Estructura esperada

```
semana-01-web/
├── index.html
├── css/
│   └── estilos.css
├── js/
│   └── app.js
└── img/
```

PASO 4 Ejecutar localmente

Puedes abrir index.html directamente o iniciar Live Server. Si usas Live Server, la URL será similar a `http://127.0.0.1:5500`. No confundas este servidor de desarrollo con un back-end de producción.

4. Caso del laboratorio

Implementaremos la página “Ruta Dev”, una pequeña guía de aprendizaje para estudiantes que empiezan desarrollo web. Tendrá encabezado, navegación, presentación, tres tarjetas informativas, una lista de buenas prácticas y un botón que alternará un consejo adicional.

4.1 Requisitos funcionales

- Mostrar el propósito del sitio y tres tecnologías fundamentales.
- Permitir navegar a secciones internas mediante enlaces.
- Mostrar u ocultar un consejo al activar un botón.
- Actualizar el texto del botón y el atributo aria-expanded.

4.2 Requisitos de calidad

- HTML semántico y un único h1.
- Diseño legible desde 320 px de ancho.
- Foco de teclado visible y contraste suficiente.
- Sin errores en consola.
- Nombres claros en español y archivos separados.

5. Construcción guiada: HTML

HTML describe el contenido mediante elementos. La indentación no cambia el resultado visual, pero revela la jerarquía y facilita encontrar errores. Cada atributo debe responder a una necesidad concreta.

index.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Ruta introductoria para aprender desarrollo web.">
  <title>Ruta Dev | Fundamentos web</title>
  <link rel="stylesheet" href="css/estilos.css">
  <script src="js/app.js" defer></script>
</head>
<body>
  <a class="salto-contenido" href="#contenido">Saltar al contenido</a>

  <header class="encabezado">
    <div class="contenedor barra-navegacion">
      <a class="marca" href="#inicio" aria-label="Ruta Dev, inicio">Ruta Dev</a>
      <nav aria-label="Navegación principal">
        <ul class="menu">
          <li><a href="#tecnologias">Tecnologías</a></li>
          <li><a href="#practicas">Buenas prácticas</a></li>
        </ul>
      </nav>
    </div>
  </header>

  <main id="contenido">
    <section id="inicio" class="hero">
      <div class="contenedor">
        <p class="eyebrow">SEMANA 1 · FUNDAMENTOS</p>
        <h1>Construye una base sólida para desarrollar en la Web</h1>
        <p class="introduccion">HTML aporta significado, CSS define la presentación y JavaScript
incorpora comportamiento.</p>
        <button id="boton-consejo" class="boton" type="button"
          aria-expanded="false" aria-controls="consejo">
          Mostrar consejo
        </button>
        <p id="consejo" class="consejo" hidden>
          Domina primero los estándares web; después comprenderás mejor cualquier framework.
        </p>
      </div>
    </section>

    <section id="tecnologias" class="seccion contenedor" aria-labelledby="titulo-tecnologias">
      <h2 id="titulo-tecnologias">Tecnologías fundamentales</h2>
      <div class="tarjetas">
        <article class="tarjeta"><h3>HTML</h3><p>Estructura el contenido y expresa su
significado.</p></article>
        <article class="tarjeta"><h3>CSS</h3><p>Controla el diseño, los colores y la
adaptación.</p></article>
        <article class="tarjeta"><h3>JavaScript</h3><p>Permite responder a acciones y modificar la
```

```

interfaz.</p></article>
</div>
</section>

<section id="practicas" class="seccion seccion-alterna">
  <div class="contenedor">
    <h2>Buenas prácticas desde el inicio</h2>
    <ul class="lista-practicas">
      <li>Usar elementos semánticos.</li><li>Separar responsabilidades.</li>
      <li>Diseñar para teclado y pantallas pequeñas.</li><li>Verificar antes de entregar.</li>
    </ul>
  </div>
</section>
</main>

<footer class="pie"><div class="contenedor"><p>Laboratorio de Marcos de Desarrollo
Web</p></div></footer>
</body>
</html>

```

5.1 Lectura razonada del HTML

<!doctype html> – indica HTML moderno.

lang="es" – ayuda a lectores de pantalla y buscadores a reconocer el idioma.

viewport – permite adaptar el ancho lógico al dispositivo.

defer – descarga el script sin bloquear el análisis del HTML y lo ejecuta cuando el DOM está disponible.

main, section, article, nav – comunican la función del contenido; no son simples reemplazos visuales de div.

button – es correcto para una acción. Un enlace se reserva para navegar.

hidden – oculta inicialmente el consejo, pero JavaScript puede cambiar ese estado.

aria-expanded y aria-controls – exponen a tecnologías de asistencia el estado y la relación del botón.

EVITA

No agregues roles ARIA que duplican la semántica nativa. Prefiere un button real antes que convertir un div en botón.

6. Construcción guiada: CSS

CSS aplica reglas mediante selectores. La cascada decide qué declaración gana considerando origen, importancia, especificidad y orden. En esta práctica utilizamos clases reutilizables, variables y diseño mobile-first.

css/estilos.css

```

:root {
  --azul-900: #0b2545; --azul-700: #174f78; --celeste-100: #eaf4f8;
  --blanco: #ffffff; --texto: #23313f; --borde: #d5e1e8;
  --sombra: 0 10px 25px rgb(11 37 69 / 12%);
}
* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body { margin: 0; color: var(--texto); background: var(--blanco); font-family: system-ui, -apple-system, "Segoe UI", sans-serif; line-height: 1.6; }
a { color: inherit; }

```

```

.contenedor { width: min(100% - 2rem, 70rem); margin-inline: auto; }
.salto-contenido { position: fixed; inset: .5rem auto auto .5rem; z-index: 10; padding: .7rem 1rem;
background: var(--blanco); color: var(--azul-900); transform: translateY(-150%); }
.salto-contenido:focus { transform: translateY(0); }
.encabezado { position: sticky; top: 0; z-index: 5; color: var(--blanco); background: var(--azul-900);
}
.barra-navegacion { display: flex; flex-wrap: wrap; align-items: center; justify-content: space-
between; gap: .75rem; padding-block: .9rem; }
.marca { font-weight: 800; text-decoration: none; }
.menu { display: flex; flex-wrap: wrap; gap: 1rem; margin: 0; padding: 0; list-style: none; }
.menu a { text-decoration: none; }
.hero { padding-block: clamp(3rem, 8vw, 6rem); color: var(--blanco); background: linear-
gradient(135deg, var(--azul-900), var(--azul-700)); }
.hero h1 { max-width: 18ch; margin-block: .4rem 1rem; font-size: clamp(2rem, 6vw, 4rem); line-height:
1.08; }
.eyebrow { margin: 0; font-size: .85rem; font-weight: 800; letter-spacing: .08em; }
.introduccion { max-width: 62ch; font-size: 1.1rem; }
.boton { border: 0; border-radius: .5rem; padding: .8rem 1rem; color: var(--azul-900); background:
var(--blanco); font: inherit; font-weight: 750; cursor: pointer; }
.boton:hover { background: var(--celeste-100); }
:focus-visible { outline: .2rem solid #ffcc4d; outline-offset: .2rem; }
.consejo { max-width: 65ch; padding: 1rem; border-left: .3rem solid #ffcc4d; background: rgb(255 255
255 / 12%); }
.seccion { padding-block: 3.5rem; }
.seccion h2 { color: var(--azul-900); font-size: clamp(1.6rem, 4vw, 2.3rem); }
.tarjetas { display: grid; gap: 1rem; }
.tarjeta { padding: 1.3rem; border: 1px solid var(--borde); border-radius: .8rem; background: var(--
blanco); box-shadow: var(--sombra); }
.tarjeta h3 { margin-top: 0; color: var(--azul-700); }
.seccion-alterna { background: var(--celeste-100); }
.lista-practicas { padding-left: 1.2rem; }
.pie { padding-block: 1.5rem; color: var(--blanco); background: var(--azul-900); text-align: center; }
@media (min-width: 48rem) { .tarjetas { grid-template-columns: repeat(3, 1fr); } }
@media (prefers-reduced-motion: reduce) { html { scroll-behavior: auto; } }

```

6.1 Decisiones de diseño

box-sizing: border-box — incluye padding y borde dentro del tamaño declarado.

min() — limita el ancho sin perder margen en pantallas pequeñas.

clamp() — crea tamaños fluidos con límites inferior y superior.

Grid — organiza las tarjetas; empieza con una columna y pasa a tres desde 48 rem.

rem — respeta mejor las preferencias de tamaño de texto que una solución basada únicamente en px.

:focus-visible — mantiene una señal clara para navegación por teclado.

prefers-reduced-motion — respeta la preferencia de reducir movimiento.

7. Construcción guiada: JavaScript

JavaScript interactúa con el DOM, la representación en memoria que el navegador construye a partir de HTML. El código espera el clic, cambia el atributo hidden y mantiene sincronizado el estado accesible del botón.

js/app.js

```

"use strict";

const botonConsejo = document.querySelector("#boton-consejo");
const consejo = document.querySelector("#consejo");

if (botonConsejo && consejo) {
  botonConsejo.addEventListener("click", () => {

```

```

const seMostrara = consejo.hidden;
consejo.hidden = !seMostrara;
botonConsejo.setAttribute("aria-expanded", String(seMostrara));
botonConsejo.textContent = seMostrara ? "Ocultar consejo" : "Mostrar consejo";
});
}

```

7.1 Explicación línea por línea

Elemento	Función
"use strict"	Activa comprobaciones más estrictas del lenguaje.
const	Declara una referencia que no será reasignada.
querySelector	Busca el primer elemento que coincide con el selector CSS.
if (...)	Evita intentar usar elementos inexistentes.
addEventListener	Registra una función que responderá al evento click.
consejo.hidden	Lee o modifica el estado oculto del elemento.
setAttribute	Sincroniza el estado accesible.
textContent	Actualiza texto sin interpretar HTML; es más seguro para texto plano.

SEGURIDAD

Evita innerHTML cuando solo necesitas insertar texto. Si en el futuro procesas datos del usuario o del servidor, no los conviertas directamente en HTML.

8. Ejecución y verificación

PASO 1 Abrir la página

Inicia Live Server o abre index.html. Comprueba que aparecen encabezado, presentación, tarjetas y pie.

PASO 2 Probar la interacción

Activa el botón. El consejo debe mostrarse, el texto debe cambiar a “Ocultar consejo” y aria-expanded debe pasar a true.

PASO 3 Probar con teclado

Pulsa Tab. El enlace para saltar al contenido debe aparecer al recibir foco. Recorre enlaces y botón; el foco debe ser visible. Activa el botón con Enter o Espacio.

PASO 4 Probar adaptación

Abre las herramientas del navegador y activa la vista de dispositivo. Prueba 320, 768 y 1440 px. No debe existir desplazamiento horizontal accidental.

PASO 5 Revisar consola

En DevTools abre Console. La entrega debe quedar sin errores. Después revisa Network para observar que index.html, estilos.css y app.js respondan correctamente.

8.1 Lista de control

- [] El título de la pestaña es descriptivo.
- [] Todos los enlaces internos llegan a la sección correcta.
- [] El diseño se adapta a móvil y escritorio.
- [] La interacción funciona con mouse y teclado.
- [] aria-expanded refleja el estado visible.
- [] No hay rutas rotas ni errores en consola.
- [] HTML, CSS y JavaScript están separados.

9. Depuración: errores frecuentes

Síntoma	Causa probable	Cómo comprobar
No se aplican estilos	Ruta incorrecta o nombre distinto	Network y href del link
El botón no responde	Script no cargado o selector incorrecto	Console, Network y los id
Se ven tres tarjetas en móvil	Media query invertida o regla base incorrecta	Computed y ancho de viewport
Aparece desplazamiento horizontal	Ancho fijo, padding o elemento desbordado	Inspector y regla overflow
El consejo aparece al inicio	Falta hidden en HTML	Inspeccionar el párrafo
Live Server no abre	Se abrió archivo, no carpeta	Reabrir carpeta completa

MÉTODO PROFESIONAL

Reproduce el error, lee el primer mensaje de consola, identifica archivo y línea, formula una hipótesis, cambia una sola cosa y vuelve a probar. No edites al azar.

10. Reto de aplicación

Extiende “Ruta Dev” sin copiar componentes externos. La meta es aplicar lo comprendido y conservar la calidad alcanzada.

Reto obligatorio

- Añade una cuarta tarjeta denominada “Herramientas del navegador”.
- Incluye una sección “Mi meta de aprendizaje” con un párrafo propio.
- Agrega un botón “Cambiar tema” que alterne entre tema claro y oscuro mediante una clase en body.
- Mantén accesible el estado del botón con aria-pressed.
- Comprueba el resultado en 320 px y con navegación por teclado.

Pistas de implementación

Pista JavaScript

```
const botonTema = document.querySelector("#boton-tema");

botonTema?.addEventListener("click", () => {
```

```
const temaOscuroActivo = document.body.classList.toggle("tema-oscuro");
botonTema.setAttribute("aria-pressed", String(temaOscuroActivo));
});
```

Pista CSS

```
.tema-oscuro {
  --blanco: #101820;
  --texto: #eef5f8;
  --borde: #3d5360;
  --celeste-100: #172832;
}
```

IMPORTANTE

La pista no incluye el botón HTML. Debes decidir su ubicación, texto, id, type y estado aria-pressed inicial. Explica tu decisión en dos o tres oraciones.

11. Evidencias de entrega

- Carpeta completa del proyecto comprimida en ZIP.
- Captura de la página en vista móvil y escritorio.
- Captura de Console sin errores.
- Breve explicación de la separación entre HTML, CSS y JavaScript.
- Respuesta: ¿qué problema resolverá Bootstrap en la siguiente semana y qué no debería reemplazar?

11.1 Convención sugerida

APELLIDO_Nombre_S01_RutaDev.zip

12. Rúbrica breve

Criterio	4 · Destacado	3 · Logrado	2 · En proceso	1 · Inicial
Estructura HTML	Semántica, jerarquía y metadatos correctos	Pequeños detalles sin afectar comprensión	Uso excesivo de div o jerarquía débil	Estructura inválida o incompleta
CSS adaptable	Mobile-first, legible y sin desbordes	Adaptación correcta con detalles menores	Fallas en uno de los anchos	Diseño rígido o ilegible
JavaScript	Interacción robusta y estado accesible	Funciona según lo pedido	Funciona parcialmente	No funciona
Calidad	Archivos organizados, nombres claros, sin errores	Un detalle menor	Varios errores o desorden	No sigue estructura
Verificación	Evidencias completas y reflexión técnica	Evidencias suficientes	Evidencias parciales	Sin comprobación

Puntaje sugerido: 20 puntos. Cada criterio vale hasta 4 puntos.

13. Preguntas de metacognición

- ¿Qué diferencia existe entre que un elemento “se vea” como botón y que sea un botón semántico?

- ¿Por qué defer resulta apropiado para este script?
- ¿Qué parte seguiría funcionando si JavaScript no se cargara?
- ¿Qué evidencia demuestra que el diseño es adaptable?
- ¿Qué beneficio y qué costo puede introducir un framework?

14. Síntesis conceptual

Concepto	Idea esencial
Cliente	Solicita recursos y presenta la interfaz.
Servidor	Recibe solicitudes y genera respuestas.
HTML	Estructura y semántica.
CSS	Presentación y adaptación.
JavaScript	Comportamiento e interacción.
Framework	Estructura y convenciones reutilizables.
Accesibilidad	La interfaz puede percibirse y operarse por más personas.
DevTools	Permite inspeccionar, medir y depurar el funcionamiento real.

15. Referencias recomendadas

- MDN Web Docs. HTML, CSS y JavaScript: <https://developer.mozilla.org/>
- WHATWG. HTML Living Standard: <https://html.spec.whatwg.org/>
- W3C. Web Accessibility Initiative: <https://www.w3.org/WAI/>
- OWASP. Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- Visual Studio Code Documentation: <https://code.visualstudio.com/docs>

CIERRE

En la semana 2 utilizarás Bootstrap. La meta no será “dejar de escribir CSS”, sino comprender cómo un sistema de componentes y utilidades acelera la construcción manteniendo semántica, accesibilidad y criterio técnico.